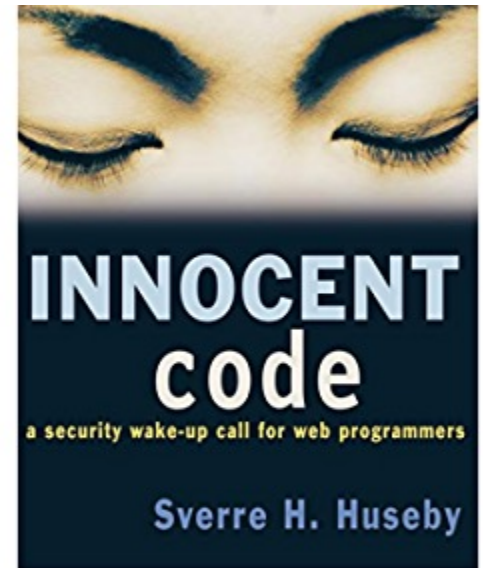

Lecture 7

Coding and Security

Outline

1. SQL Injection
2. Output Handling
-
3. Programming Languages Problem
4. Java Security
5. Java Security Demo



★★★★★ 5

The book “Innocent Code” by Sverre Huseby has many useful examples.

JAVA Security Packages

JAVA Standard Crypto Libraries

- It is a bad idea to implement our own crypto algorithms.
- Building a secure system may need to use standard crypto algorithms.
- We do this by using a library of supplied crypto implementations.
- Java has its own crypto packages:
 - `java.security.*`
 - `javax.crypto.*`
 - `javax.crypto.spec.*`

Crypto Libraries: Interfaces and Implementations

- These libraries are implemented in the standard way.
- All functionality is provided in interfaces.
- Actual implementations are provided by factory methods.
- This makes it easy to switch implementations.
 - The bulk of the code uses the interfaces, which will not change.
 - Just the factory method needs to change to select a different implementation.

Crypto Libraries: Interfaces and Implementations

- I will cover examples of cryptographic functions using the standard Java crypto libraries
 - An implementation of some cryptographic functions in the file `MainJavaCrypto.java`.
 - Check it out on Moodle
- I will then illustrate this with a program I wrote called `FileEncryptionApp`.
 - The project called `FileEncryption`. Check it out on Moodle

Java Cryptographic Functions: Implementation

JAVA Cryptographic Functions

- Symmetric key generation.
 - Getting Java to provide a random AES key.
 - This could be used for the encryption / decryption.
- Public Key Encryption
 - Creating a pair of RSA keys.
 - Encrypting and decrypting with these keys.
- Message Digests
- Digital signatures
 - Signing a message digest and verifying that it is correct.
- The older functionality is in `java.security`
- The later in `javax.crypto`

AES Key Generation

- From `javax.crypto` library.
- `KeyGenerator` is used to generate a `SecretKey`.
 - There is only one key for AES encryption and decryption and so it must be kept secret.
- It is generated by the `getInstance` factory method, specifying AES.
- The key length is provided in the `init` method.
- The key is created by the `generateKey` method.
- `MyPrintBytes` method will print out a byte array in hexadecimal.
- There is an exception to be caught (see full listing).

AES Key Generation: Code

```
SecretKey aes_key = null;
```

```
..
```

```
KeyGenerator kg = KeyGenerator.getInstance("AES");
```

```
kg.init(128); // key length 128
```

```
aes_key = kg.generateKey();
```

AES Encryption/Decryption

- `javax.crypto` has an abstract class called `Cipher`.
- Use the `getInstance` factory method specifying an AES cipher.
- Initialise it by calling the `init` method with the mode and key as parameters.
 - There are 2 possible modes.
 - `Cipher.ENCRYPT_MODE`
 - `Cipher.DECRYPT_MODE`
- Finally, the encryption is done by the `doFinal` method.
- Long files need several calls to `update` before `doFinal`.
- There are some exceptions (not included for brevity).

AES Encryption/Decryption: Code

```
import java.security.*;  
Cipher c = null;  
c = Cipher.getInstance("AES");  
c.init(mode, k);  
return c.doFinal(in);
```

- The ciphertext / decrypted text is returned as an array of bytes.
- I chose not to define an initialisation vector for simplicity
- `new GCMParameterSpec(s.getTLen(), IV)`
where IV is an array of bytes.

RSA Keypair Generation

- From the earlier `java.security` library.
 - It uses `initialize` instead of `init`!
- `KeyPairGenerator` is created by the `getInstance` factory method.
 - `initialize` must be called with the key length in bits.
 - It will then generate a `KeyPair`
- The `KeyPair` object will then provide `PublicKey` and `PrivateKey` objects.
 - These keys include exponent and modulus and are much longer than the AES key.
 - They also take significantly longer to create.

RSA Keypair Generation: Code

```
PublicKey pk = null;
```

```
PrivateKey sk = null;
```

```
...
```

```
KeyPairGenerator kg =
```

```
KeyPairGenerator.getInstance("RSA");
```

```
kg.initialize(2048);
```

```
KeyPair kp = kg.generateKeyPair();
```

```
pk = kp.getPublic();
```

```
sk = kp.getPrivate();
```

RSA Encryption/Decryption

- This is very similar to encrypting with AES.
 - `getInstance` specifies RSA.
- There are a lot of possible exceptions to catch!
- Parameters `mode`, `key` and `input` (array of bytes) must be provided
- An array of bytes is produced.

```
Cipher c = null;  
.  
.  
.  
c = Cipher.getInstance("RSA");  
c.init(mode, key);  
return c.doFinal(in);
```

Message Digest

- Again, from the earlier `java.security` library.
- A `MessageDigest` is created by the `getInstance` method
 - I have chosen the SHA-256 message digest.
- `update` adds an array of bytes.
 - Many updates can be added for long inputs.
 - For example, a file can be read in chunks, each updated.
 - The array of bytes can be reused, so that the whole file does not have to be read into memory first.
- The `digest` method returns an array of bytes, the method digest.

Message Digest: Code

```
String mdin = new String("long message to  
be hashed!");  
byte[] digest = null;  
.  
.  
.  
MessageDigest md = MessageDigest  
    .getInstance("SHA-256");  
md.update(mdin.getBytes());  
digest = md.digest();
```

Digital Signature

- The standard way of using digital signatures is to encrypt a message digest with a secret key.
- The signature is verified by decrypting the signature with the public key and checking that the revealed message digest is the same as a message digest calculated from the same input.
- In the code, note that we encrypt with the secret key and decrypt with the public key.
- The `isEqual` method of `MessageDigest` compares two message digests.

Digital Signature: Code

```
// sign the digest
byte[] signature =
    rsa_encrypt(Cipher.ENCRYPT_MODE, sk, digest);
// undo the signature
byte[] undosig =
    rsa_decrypt(Cipher.DECRYPT_MODE, pk, signature);
// compare the two
if (MessageDigest.isEqual(digest, undosig))
    System.err.println("equal");
else
    System.err.println("DIFFERENT!!!");
```

File Encryption App: Implementation

File Encryption Application

- This program encrypts a number of small files.
 - They will mainly be text files.
- The user provides the encryption key, and the Java libraries are used to do the encryption.
- Related files are grouped together in an index, which just produces one crypto file.
 - This index is compressed before being encrypted.
- Plaintext files are created when they are opened.
- The plaintext files are erased and then deleted when they are closed or the application closes.

Key Generation from User Input

- Encryption scheme: `128 x 128 AES`.
- The key is provided by a user as a `String` (`<= 16` characters).
- Be converted into a `128-bit key` suitable for AES.
 - This is 16 bytes.
 - The key is padded to 16 characters by appending `'z'` characters.
- `Javax.crypto.spec` has an abstract class (interface) called `Key`.
- The implementation class `SecretKeySpec` is used to create an actual `Key` object.
 - The first parameter is an array of 16 bytes, from the `String`.
 - The second parameter specifies “AES”.

Key Generation: Code

```
import javax.crypto.spec.*;

. . .
String key

. . .
// pad the key to 16 characters
int klen = key.length();
for (int i = klen; i < 16; i++)
    key += 'z';

// initialise the key Object
Key k = new SecretKeySpec(key.getBytes(), "AES");
```

AES Encryption/Decryption

- `javax.crypto` has an abstract class called `Cipher`.
- Use the `getInstance` factory method specifying an AES cipher.
- Initialise it by calling the `init` method with the mode and key as parameters.
 - There are 2 possible modes.
 - `Cipher.ENCRYPT_MODE`
 - `Cipher.DECRYPT_MODE`
- Finally, the encryption is done by the `doFinal` method.
- Long files need several calls to `update` before `doFinal`.
- There are some exceptions (not included for brevity).

AES Encryption/Decryption: Code

```
import java.security.*;
Cipher c = null;
c = Cipher.getInstance("AES");
c.init(mode, k);
return c.doFinal(in);
```

- The ciphertext / decrypted text is returned as an array of bytes.
- I chose not to define an initialisation vector for simplicity
- `new GCMParameterSpec(s.getTLen(), IV)`
where IV is an array of bytes.

Compression

- Compression is done with the `java.util.zip` library
- Compression is done with a `Deflater` object and decompression with an `Inflater` object.
- I store the length in 4 bytes at the front so that I can create an array of the right size when uncompressing.

Compression/Decompression: Code

```
Deflater d = new Deflater();  
d.setInput(b);  
d.finish();  
byte[] tmp = new byte[b.length];  
int n = d.deflate(tmp);
```

```
Inflater u = new Inflater();  
u.setInput(b, 4, b.length-4);  
byte[] out = new byte[len];  
u.inflate(out);  
u.end();
```

File Erase before Deleting

- Deleting a file does not delete the contents.
 - The disk blocks making up the file are marked for recycling.
- I create an array of bytes that is the same length of the file and fill it with random values.
- I then write all the bytes to the file, overwriting the content with random values.
- Finally, I delete the file.
- The code is in class `Entry`.

File Erase: Code

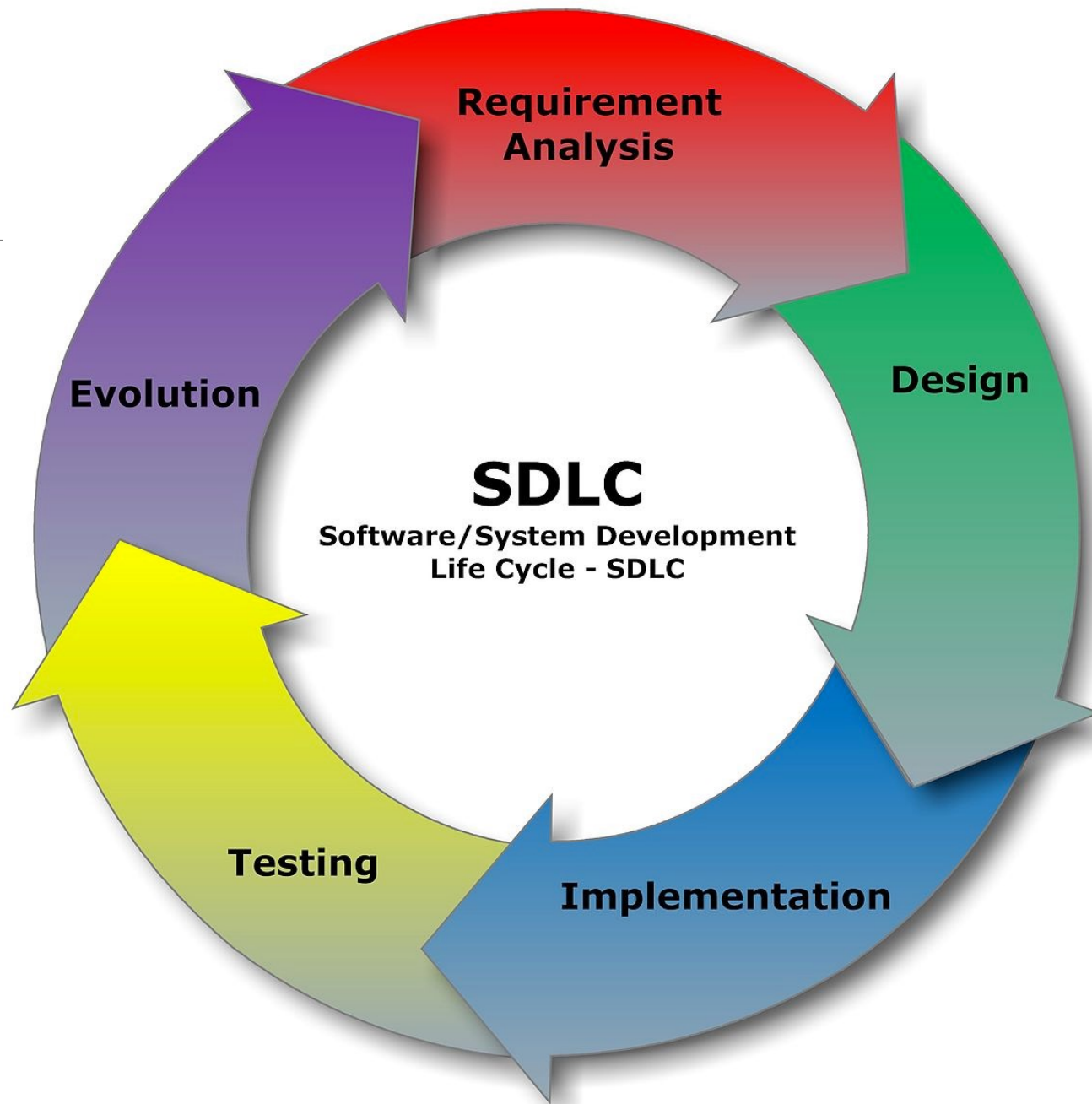
```
// erase content of real file
```

```
Random rand = new Random();  
byte[] b = new byte[(int)f.length()];  
rand.nextBytes(b);  
FileIO.write(f, b);
```

```
// delete real file
```

```
f.delete();  
f = null;
```

A big picture of Secure Development



Secure Software Development

Consider security throughout the Software Development LifeCycle

- Requirements
- Design
- Implementation
- Testing
- Deployment

Requirements

Identify sensitive data and resources

Define security requirements for them

- Confidentiality
- Integrity
- Availability

Consider threats and abuse cases that violate these requirements

Application Specific

- Abuse/Misuse Cases
- Threat Models
- Attacks
- Assets

Generic

- Common Best Practices
- Legal
- IT
- Development

Architectural Risk Analysis

- Underlying Framework
- Ambiguity Analysis
- Fundamental Weakness

Attack Patterns

- Historical Risks
- Vulnerabilities

Design

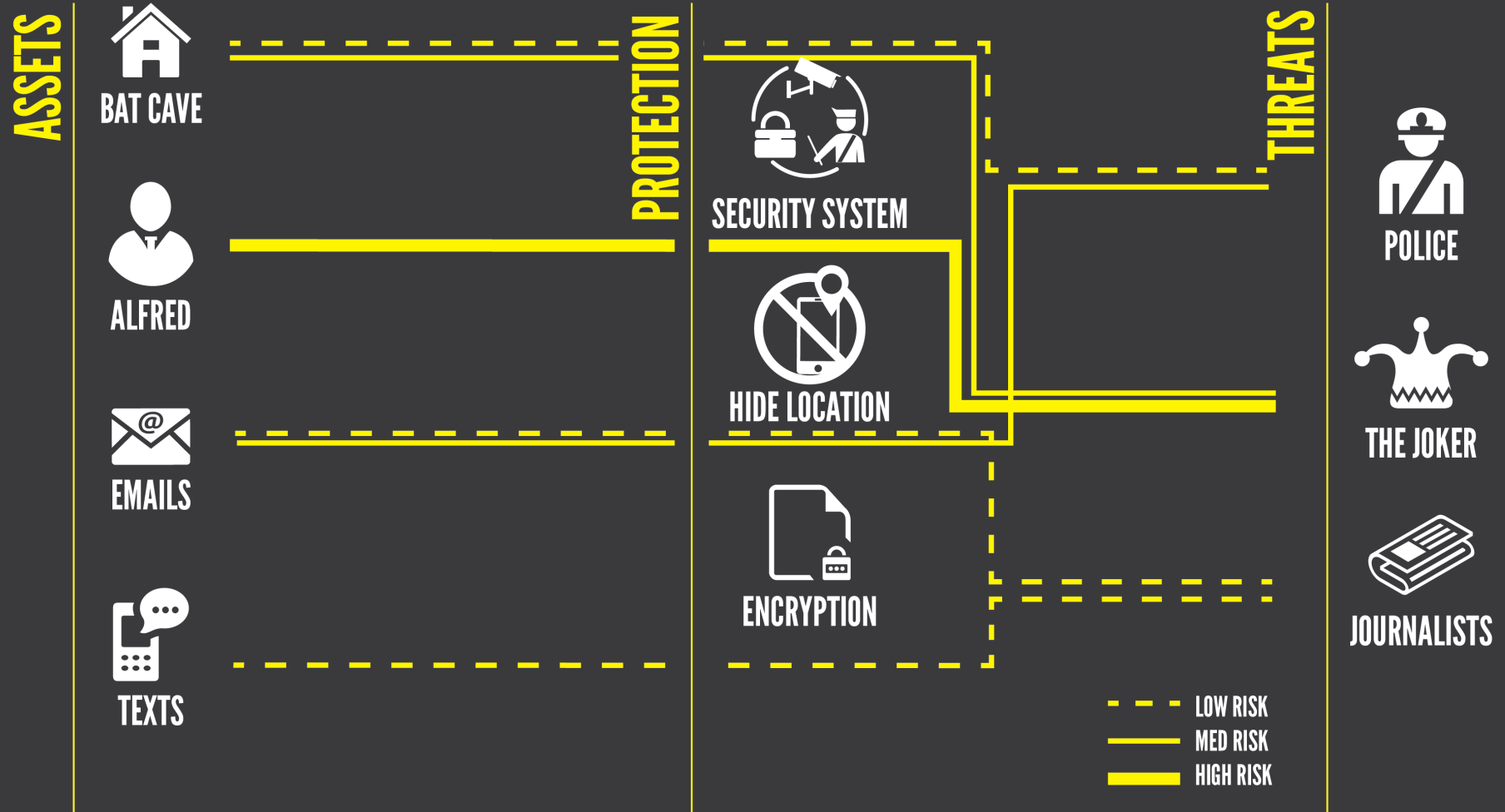
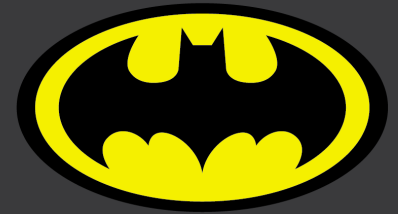
Apply principles for secure software design

- Prevent, mitigate and detect possible attacks

Security principles

- Favor Simplicity
- Trust with Reluctance
- Defend in Depth

BRUCE WAYNE/BATMAN'S THREAT MODEL



Implementation

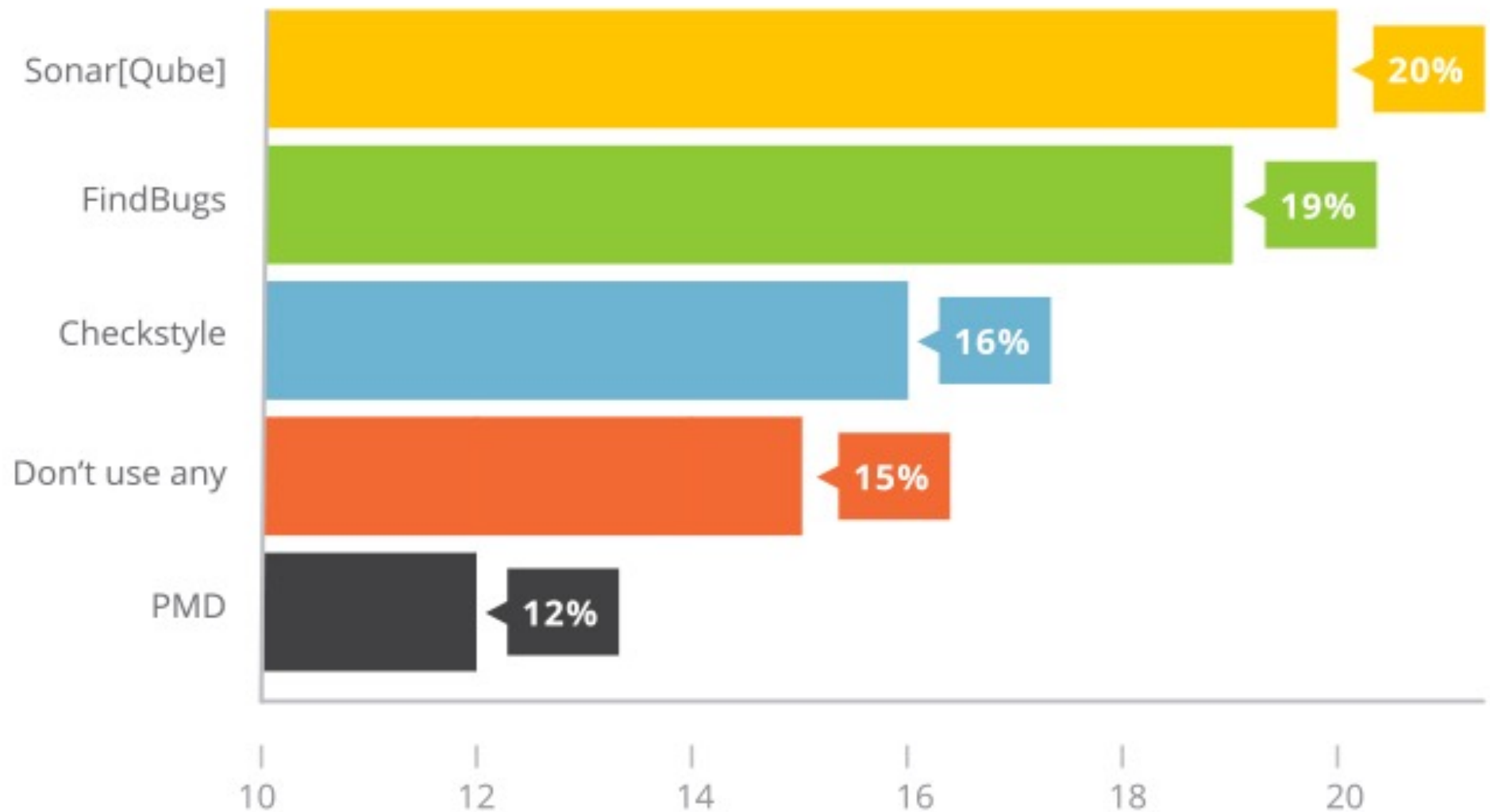
Apply coding rules that implement secure design

Use automated code review techniques to find potential vulnerabilities components

- Static Analysis
- Symbolic execution

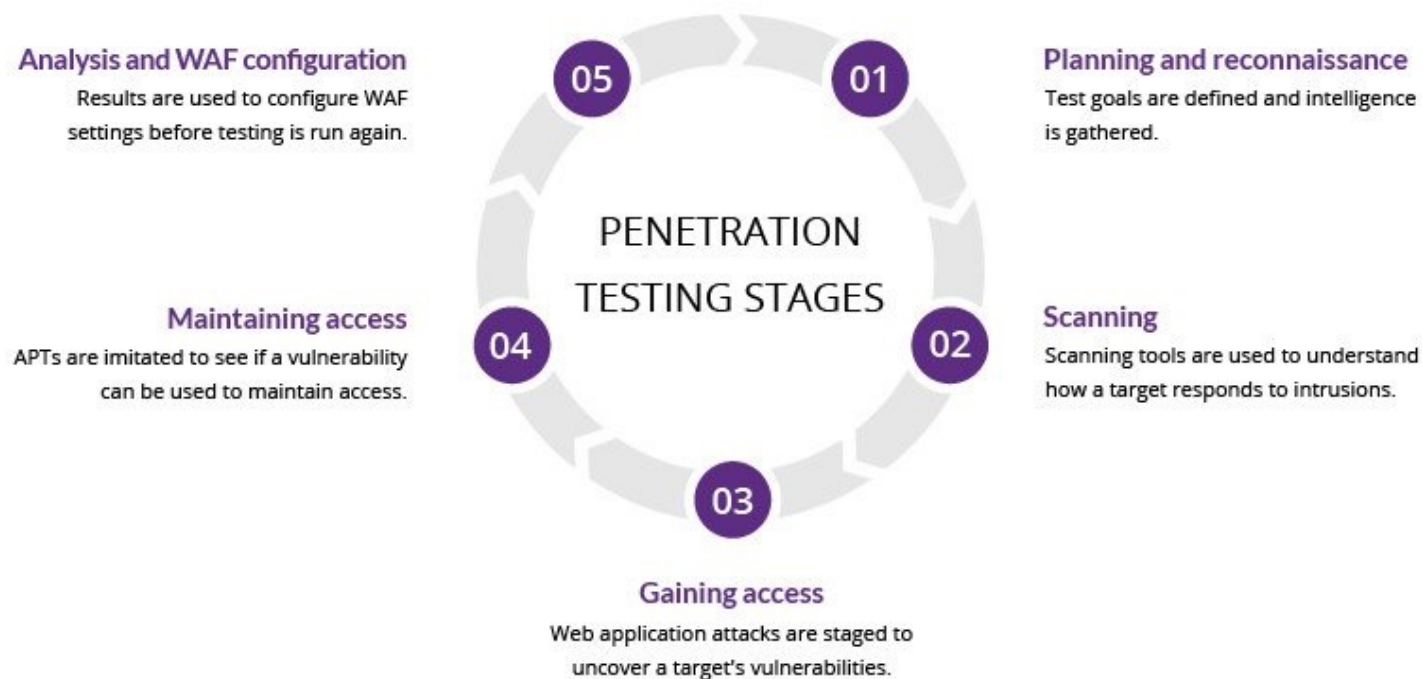
STATIC CODE ANALYSIS TOOL USAGE BY DEVELOPERS

(SAMPLE SIZE: 2119)



Testing

1. Penetration Testing to find potential flaws in the real system
 - Fuzz testing
 - Employ attack patterns



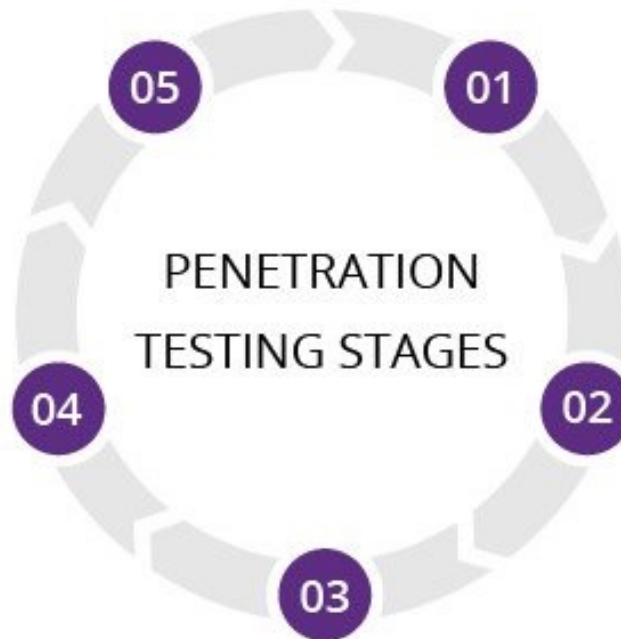
Penetration Testing

Analysis and WAF configuration

Results are used to configure WAF settings before testing is run again.

Maintaining access

APTs are imitated to see if a vulnerability can be used to maintain access.



Planning and reconnaissance

Test goals are defined and intelligence is gathered.

Scanning

Scanning tools are used to understand how a target responds to intrusions.

Gaining access

Web application attacks are staged to uncover a target's vulnerabilities.

Different Testing Methodologies

2. BSIMM (Building Security In – Maturity Model)

- <http://bsimm.com>

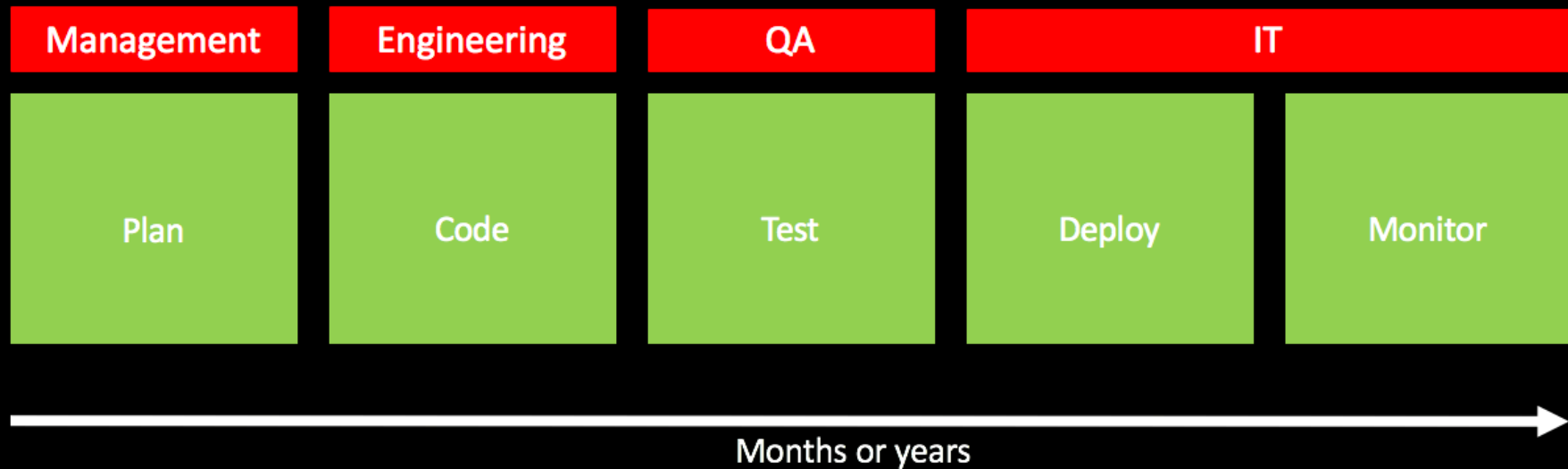
3. Microsoft Security Development Lifecycle

- <https://www.microsoft.com/en-us/sdl/>

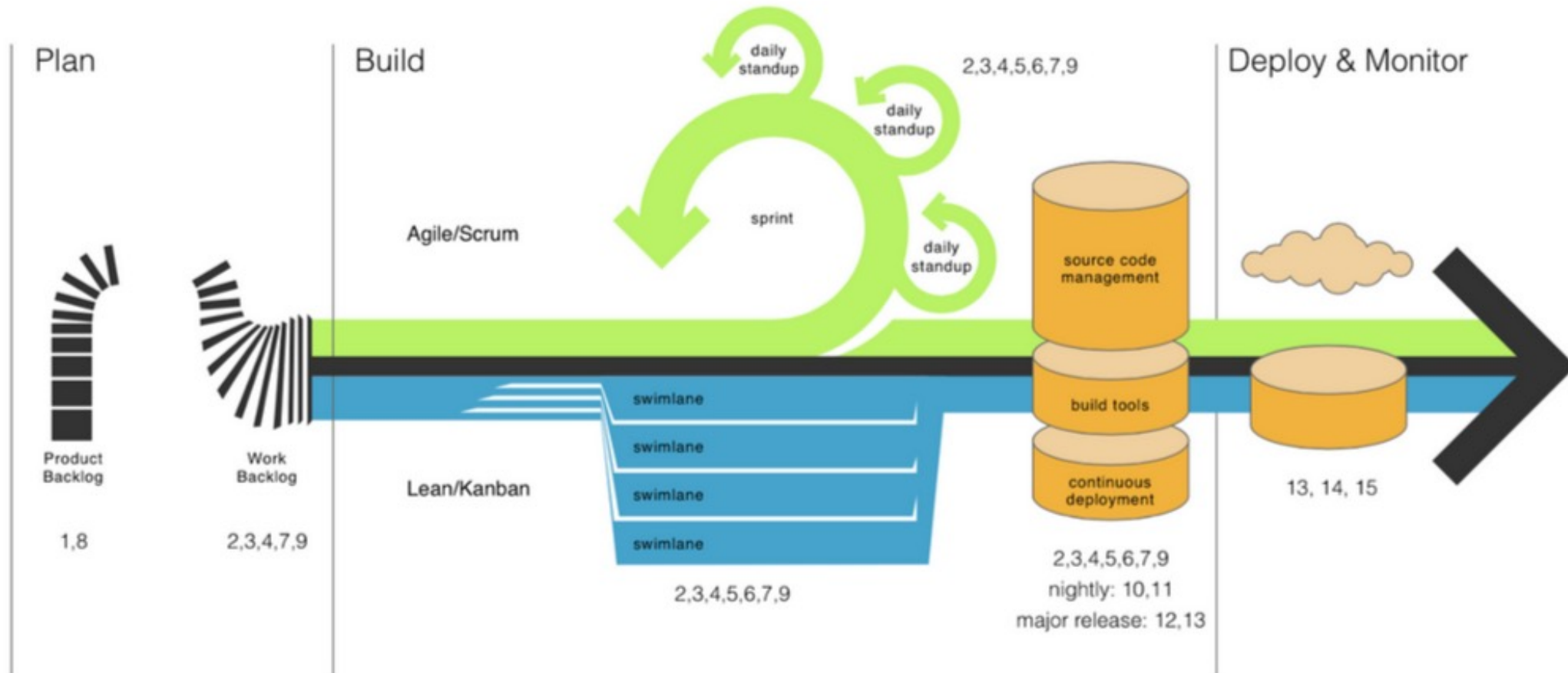
4. OpenSAMM Software Assurance Maturity Model

- <http://opensamm.org>

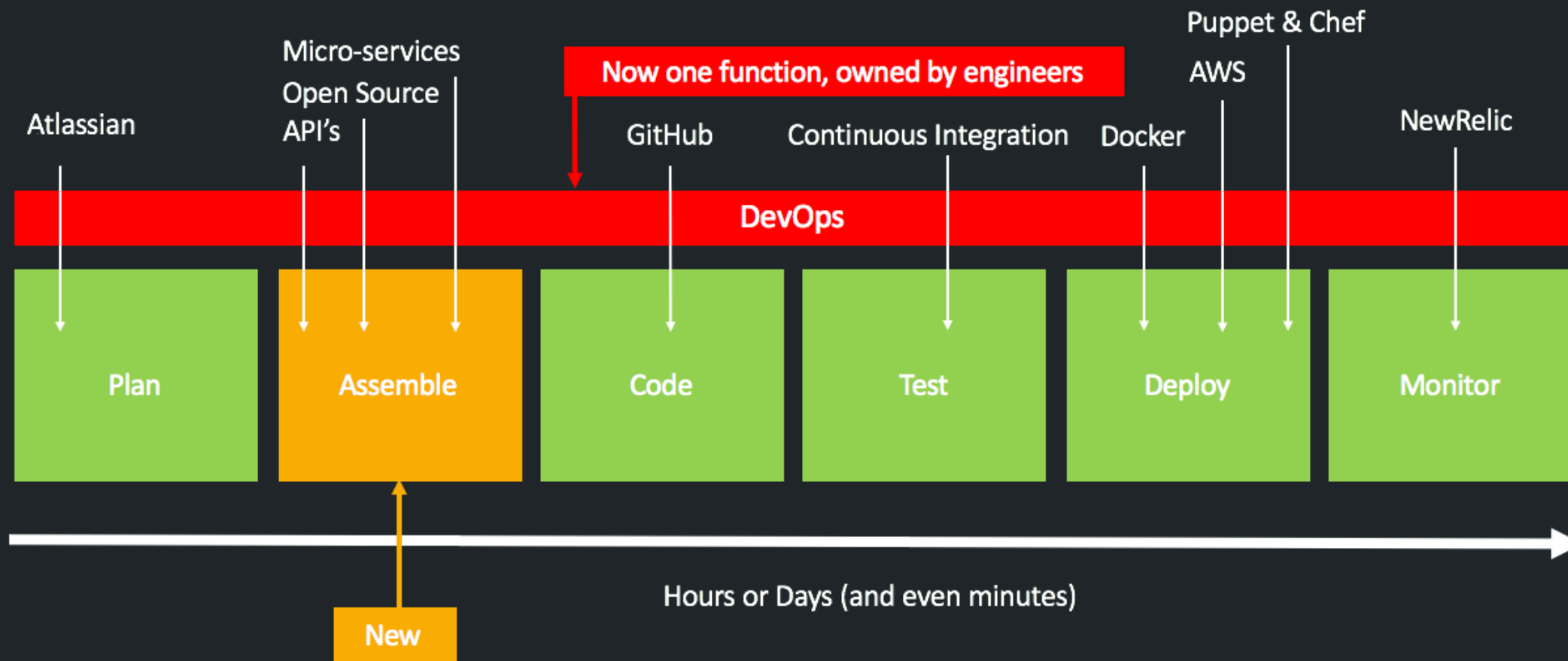
Old School



Continuous Delivery of Software



New school



[:]

Continuous Security

Requires security automation

Integrate into CD environment and tools

- Source code management systems
 - GitHub, Bitbucket etc.
- Build systems
 - Travis CI, Jenkins etc.

Audit third party component and open-source library usage

Takeaways

1. Security practices should be built in during the software development process.
2. Continuous delivery needs continuous security.